講義 16：Field-Symbol（指向資料物件的別名，不是複本）（授課順序：接在講義 5 之後）

> 對應練習：ex16 | 答案程式：`ZR_TR16_FIELD_SYMBOLS`

本講重點

- Field-Symbol 的本質：**別名（指向本尊），不是複本**
- `FIELD-SYMBOLS` 宣告與 `ASSIGN`
- `LOOP AT ... ASSIGNING` / `READ TABLE ... ASSIGNING`：就地修改內表（Internal Table）
- `ASSIGN COMPONENT ... OF STRUCTURE`：動態逐欄存取
- `IS ASSIGNED` / `UNASSIGN` 與執行期錯誤（Runtime Error） GETWA_NOT_ASSIGNED

1. 本質：別名不是複本

上一講的痛點：`LOOP AT ... INTO` 拿到的是**複本**，改完要 MODIFY 寫回，忘了就白改。Field-Symbol 換一種思路——不複製資料，而是給既有資料物件取一個**別名**：ASSIGN 之後，`<fs>` 就是那個變數「本人」，改 `<fs>` 等於改本尊。

```
DATA gv_total TYPE i VALUE 100.
FIELD-SYMBOLS <fs_num> TYPE i.

ASSIGN gv_total TO <fs_num>.
<fs_num> = <fs_num> + 50.
WRITE / gv_total.          " 150！改別名就是改本尊
```

	work area（INTO）	**field-symbol（ASSIGNING）**
拿到什麼	複本	本尊的別名
改了之後	要 MODIFY 寫回	直接生效
大表效能	每筆複製一次	不複製，較快

2. 宣告與 ASSIGN

```
FIELD-SYMBOLS <fs_num>  TYPE i.            " 只能指向 i 型別
FIELD-SYMBOLS <fs_stu>  TYPE ty_student.   " 只能指向該結構
FIELD-SYMBOLS <fs_any>  TYPE any.          " 什麼都能指（動態場景用）
```

- 名稱**必須**用角括號 `<...>` 包起來，慣例 `<fs_...>` 或 `<ls_...>` / `<lv_...>`。
- 宣告後、ASSIGN 前，field-symbol 處於「未指派」狀態——這時候使用它會**執行期當掉**（GETWA_NOT_ASSIGNED），所以有不確定時先問 `IS ASSIGNED`：

```
ASSIGN gv_total TO <fs_num>.
IF <fs_num> IS ASSIGNED.
  <fs_num> = 999.
ENDIF.

UNASSIGN <fs_num>.            " 解除指派，回到未指派狀態
```

3. LOOP AT ... ASSIGNING：就地修改內表

這是 field-symbol 最重要的應用。對照講義 5 的「INTO + MODIFY」：

```
* 舊寫法：複本 + 寫回
LOOP AT gt_students INTO gs_student.
  gs_student-grade = 'A'.
  MODIFY gt_students FROM gs_student.    " 忘了這行就白改
ENDLOOP.

* field-symbol 寫法：直接改表格那一列，沒有 MODIFY
FIELD-SYMBOLS <ls_student> TYPE ty_student.    " 先宣告：指向內表的「一列」，型別跟那一列一樣

LOOP AT gt_students ASSIGNING <ls_student>.
  IF <ls_student>-score >= 80.
    <ls_student>-grade = 'A'.
  ELSEIF <ls_student>-score >= 60.
    <ls_student>-grade = 'B'.
  ELSE.
    <ls_student>-grade = 'C'.
  ENDIF.
ENDLOOP.
```

上面是傳統寫法：**先用 `FIELD-SYMBOLS` 宣告，再 `ASSIGNING <ls_student>`**。7.40 之後的行內宣告（Inline Declaration）寫法 `ASSIGNING FIELD-SYMBOL(<ls_student>)`，跟本講其他用法的新寫法一起整理在第 7 節。

READ TABLE 也有對應版本，單筆就地修改：

```
FIELD-SYMBOLS <ls_hit> TYPE ty_student.

READ TABLE gt_students ASSIGNING <ls_hit> WITH KEY id = 'S0004'.
IF sy-subrc = 0.
  <ls_hit>-score = <ls_hit>-score + 30.      " 直接改表中那筆
ENDIF.
```

> sy-subrc 鐵律不變：READ ASSIGNING 失敗時 field-symbol 維持先前狀態，不檢查就用，輕則改錯筆、重則當掉。

4. ASSIGN COMPONENT：動態逐欄存取

不知道（或不想寫死）欄位名時，可以用「第幾欄」動態取欄位——維護舊程式、寫泛用工具（如萬用匯出）時常見：

```
FIELD-SYMBOLS <fs_comp> TYPE any.        " any：每一欄的型別都不同，只有 any 都能接

READ TABLE gt_students INTO gs_student INDEX 1.      " 第一筆：S0001 王小明 85 分（等第 A）
DO.
  ASSIGN COMPONENT sy-index OF STRUCTURE gs_student TO <fs_comp>.
  IF sy-subrc <> 0.         " 欄位要完了：sy-index 超過欄位數
    EXIT.
  ENDIF.
  WRITE: / '欄位', sy-index, ':', <fs_comp>.
ENDDO.
* ty_student 共 4 欄（id / name / score / grade），DO 迴圈跑 5 圈，WRITE 依序輸出：
*   第 1 圈  sy-index = 1 → <fs_comp> 指向 id     →   欄位 1 : S0001
*   第 2 圈  sy-index = 2 → <fs_comp> 指向 name   →   欄位 2 : 王小明
*   第 3 圈  sy-index = 3 → <fs_comp> 指向 score  →   欄位 3 :        85
*   第 4 圈  sy-index = 4 → <fs_comp> 指向 grade  →   欄位 4 : A
*   第 5 圈  sy-index = 5 → 沒有第 5 欄，ASSIGN 失敗 sy-subrc <> 0 → EXIT，這一圈不會 WRITE
```

- `<fs_comp>` 宣告成 `TYPE any`：同一個 field-symbol 每圈被重新指派到**不同型別**的欄位（c / string / i / c），沒有任何單一具體型別接得住，所以要用 `any`。
- `COMPONENT` 後面可以是欄位序號（1 起算），也可以是**欄位名**（字元變數），兩種都常見。
- 搭配 `DO` + `sy-index`，要不到欄位（sy-subrc <> 0）就 EXIT，是走訪未知結構的標準套路。

5. 什麼時候用 Field-Symbol

- **要改內表內容**：LOOP ASSIGNING / READ ASSIGNING，勝過 INTO + MODIFY（不會忘寫回、大表更快）。
- **只是讀**：INTO 或 ASSIGNING 都可以；大表（萬筆級）建議 ASSIGNING 省複製成本。
- **動態欄位**：ASSIGN COMPONENT 幾乎是唯一選擇。
- 注意：ASSIGNING 拿到的是本尊，**手滑改到就真的改了**——純讀取的迴圈裡不要順手對 `<fs>` 賦值。

6. 常見錯誤與陷阱

症狀	原因
執行期當掉 GETWA_NOT_ASSIGNED	未 ASSIGN（或已 UNASSIGN）就使用 field-symbol
LOOP ASSIGNING 裡「順手」改了值，資料被污染	別名直通本尊，讀取用途不要賦值
ASSIGN 之後 IS ASSIGNED 還是 false	ASSIGN 失敗（如 COMPONENT 超出範圍），檢查 sy-subrc

症狀	原因
迴圈外繼續用 `<ls_student>`	迴圈結束後仍指向最後一列，語意易錯——迴圈外別再用
對 field-symbol 宣告 VALUE	field-symbol 不是變數，不能有初始值

7. 新式寫法對照（7.40 之後）：`FIELD-SYMBOL( )` 與 `ASSIGN itab[ ]`

前面全部用傳統寫法：**先 `FIELD-SYMBOLS` 宣告、再 `ASSIGNING`**。7.40 之後，field-symbol 也能在「第一次使用的地方」直接宣告，寫成 `FIELD-SYMBOL(<名稱>)`，跟變數的 `DATA(名稱)`（講義 2）是同一種**行內宣告**。本講各種用法的新舊寫法並排對照：

用法	傳統寫法（本課程先用這個）	行內宣告新寫法
LOOP ASSIGNING	`FIELD-SYMBOLS <ls_student> TYPE ty_student.` `LOOP AT gt_students ASSIGNING <ls_student>.`	`LOOP AT gt_students ASSIGNING FIELD-SYMBOL(<ls_student>).`
READ TABLE ASSIGNING	`FIELD-SYMBOLS <ls_hit> TYPE ty_student.` `READ TABLE gt_students ASSIGNING <ls_hit> WITH KEY id = 'S0004'.`	`READ TABLE gt_students ASSIGNING FIELD-SYMBOL(<ls_hit>) WITH KEY id = 'S0004'.`
ASSIGN COMPONENT	`FIELD-SYMBOLS <fs_comp> TYPE any.` `ASSIGN COMPONENT sy-index OF STRUCTURE gs_student TO <fs_comp>.`	`ASSIGN COMPONENT sy-index OF STRUCTURE gs_student TO FIELD-SYMBOL(<fs_comp>).`
指向表格的某一列	`READ TABLE gt_students ASSIGNING <ls_hit> WITH KEY id = 'S0004'.`	`ASSIGN gt_students[ id = 'S0004' ] TO FIELD-SYMBOL(<ls_hit>).`

最後一列用的是講義 4 的表格運算式（Table Expression）`itab[ ]`。跟直接取值不同的是：**`ASSIGN` 一個找不到的列不會發生例外，而是設定 `sy-subrc = 4`**，所以後面照樣用 `IF sy-subrc = 0.` 判斷即可（`ZR_TR16_FIELD_SYMBOLS_NEW` 實測）。

```
* 行內宣告版的完整範例：少了獨立的 FIELD-SYMBOLS 宣告，其餘邏輯完全相同
LOOP AT gt_students ASSIGNING FIELD-SYMBOL(<ls_student>).
  IF <ls_student>-score >= 80.
    <ls_student>-grade = 'A'.
  ENDIF.
ENDLOOP.
```

行內宣告的三個重點：

- **型別自動推導**：`LOOP` / `READ TABLE` 的 `<ls_...>` 型別由內表的列決定，不用自己寫 `TYPE ty_student`；`ASSIGN COMPONENT` 因為每圈指到的欄位型別都不同，推導出來的是 `any`，跟傳統寫法宣告的 `TYPE any` 一致。
- **作用範圍是整個程式單元，不是只有 LOOP 裡面**：行內宣告出來的 field-symbol，在同一個 FORM / 方法 / 程式主體內都看得到；因此**同一個程式單元裡，同一個名稱不能行內宣告兩次**（例如兩個 LOOP 都寫 `ASSIGNING FIELD-SYMBOL(<ls_row>)` 會編譯錯誤「已經宣告過」），要嘛換不同名稱，要嘛改成先宣告一次、後面的 LOOP 直接 `ASSIGNING <ls_row>`。
- **兩種寫法效果完全相同**，不影響執行結果與效能；差別只是少寫一行宣告。新專案 / S/4HANA 程式碼常見行內宣告，維護舊程式則要認得傳統寫法，兩種都要看得懂。

練習 16 的新式寫法版本見答案程式 `ZR_TR16_FIELD_SYMBOLS_NEW`（輸出跟原版逐行比對相同）：用 `FIELD-SYMBOL( )` 行內宣告、`ASSIGN itab[ ]` 取代 `READ TABLE ... ASSIGNING`、`COND` 打等第（講義 17）、`VALUE` 建資料（講義 4）。

8. 課堂練習

完成 ex16：驗證別名行為、LOOP ASSIGNING 打等第、READ ASSIGNING 補分、ASSIGN COMPONENT 逐欄輸出，並實驗 UNASSIGN 後使用的執行期錯誤。